

Guía Pedagógica: Día 7 - Vista Profunda, RxJS (forkJoin) y Async Pipe

Hoy llevaremos nuestra página de detalles al siguiente nivel. Aprenderemos uno de los conceptos más poderosos e importantes en el ecosistema Angular: el manejo de flujos asíncronos complejos utilizando la librería **RxJS** y conectándolos directamente a nuestro HTML con el **Async Pipe**.

Además, practicaremos la **Composición de Componentes** creando tarjetas reutilizables para los actores del elenco.

Contexto Pedagógico: El Problema de las Múltiples Peticiones

Imagina que estás en un restaurante.

1. Pides tu comida principal (Detalles de la Película).
2. Pides tu bebida (Elenco de Actores).

Podrías esperar a que llegue la comida para recién pedir la bebida (Peticiones en Serie), pero eso sería muy lento. Lo ideal es pedir ambas cosas al mismo tiempo al mesero y que te las traigan juntas (Peticiones en Paralelo).

En Angular, la herramienta perfecta para hacer múltiples peticiones HTTP al mismo tiempo y esperar a que TODAS terminen antes de mostrar la página es el operador **forkJoin** de RxJS.

Paso 1: Nuevos Modelos de Datos

Nuestra aplicación necesita saber qué forma tienen los nuevos datos que vamos a recibir de la API.

1. Actualizar el Modelo de Película

Abre **src/app/core/models/movie.model.ts** y añade los nuevos campos que la API nos envía cuando pedimos el detalle completo:

```
export interface Movie {  
  // ... (campos anteriores)  
  runtime?: number; // Duración en minutos  
  genres?: { id: number; name: string }[]; // Lista de géneros (Ej: Acción, Comedia)  
}
```

2. Crear el Modelo de Elenco (Cast)

Crema un nuevo archivo `src/app/core/models/cast.model.ts`:

```
export interface CastMember {  
  id: number;  
  name: string;  
  character: string; // El nombre del personaje en la película  
  profile_path: string | null; // La foto del actor  
}  
  
export interface CreditsResponse {  
  id: number;  
  cast: CastMember[]; // La lista de actores  
}
```

Paso 2: Ampliando el Servicio (El Mensajero)

Abre `src/app/core/services/movie.service.ts` y añade el método para pedir los actores. Asegúrate de importar `CreditsResponse`.

```
// Importa el nuevo modelo  
import { CreditsResponse } from '../models/cast.model';  
  
// Dentro de la clase MovieService:  
getMovieCredits(id: string | number) {  
  return this.http.get<CreditsResponse>  
    (`${this.apiUrl}/movie/${id}/credits`);  
}
```

Paso 3: Composición - La Tarjeta del Actor (CastCard)

Vamos a crear un componente pequeño y "Dumb" (tonto) cuya única responsabilidad sea mostrar la foto y el nombre de un actor.

1. Generar el componente

En tu terminal:

```
ng g c shared/components/cast-card
```

2. La Lógica (TypeScript)

Abre cast-card.ts y añade el @Input para recibir los datos del actor.

```
import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { CastMember } from '../../../core/models/cast.model';

@Component({
  selector: 'app-cast-card',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './cast-card.html',
  styleUrls: ['./cast-card.css']
})
export class CastCard {
  @Input({ required: true }) actor!: CastMember;

  get profileUrl() {
    return this.actor.profile_path
      ? `https://image.tmdb.org/t/p/w200${this.actor.profile_path}`
      : 'assets/no-profile.png';
  }
}
```

3. La Vista (HTML)

En cast-card.html:

```
<div class="cast-card">
  <div class="profile-container">
    <img [src]="profileUrl" [alt]="actor.name" class="profile-
image" loading="lazy">
  </div>
  <div class="actor-info">
    <h4 class="actor-name">{{ actor.name }}</h4>
    <p class="actor-character">{{ actor.character }}</p>
  </div>
</div>
```

4. Los Estilos (CSS)

En `cast-card.css` aplicaremos estilos para que parezca una tarjeta pequeña y elegante.

```
.cast-card {
  width: 140px;
  flex: 0 0 auto;
  border-radius: 8px;
  overflow: hidden;
  background-color: #1a1a1a;
  box-shadow: 0 4px 6px rgba(0,0,0,0.3);
  transition: transform 0.2s;
}

.cast-card:hover {
  transform: translateY(-5px);
}

.profile-container {
  aspect-ratio: 2 / 3;
  width: 100%;
}

.profile-image {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.actor-info {
  padding: 0.8rem;
}

.actor-name {
  font-size: 0.9rem;
  font-weight: bold;
  margin: 0 0 0.3rem 0;
  color: #fff;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

.actor-character {
  font-size: 0.8rem;
  margin: 0;
  color: #aaa;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

Paso 4: Magia con RxJS y Async Pipe en MovieDetails

Aquí es donde todo se conecta de forma elegante.

1. La Lógica Combinada (TypeScript)

Abre **movie-details.ts**. Vamos a cambiar los Signals por un **Observable** combinado.

```
import { Component, inject, Input, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MovieService } from '../../core/services/movie.service';
import { Movie } from '../../core/models/movie.model';
import { CastCard } from '../../shared/components/cast-card/cast-card';
import { Observable, forkJoin } from 'rxjs'; // Importamos RxJS
import { CreditsResponse } from '../../core/models/cast.model';

@Component({
  selector: 'app-movie-details',
  standalone: true,
  imports: [CommonModule, CastCard], // No olvides importar CastCard
  templateUrl: './movie-details.html',
  styleUrls: ['./movie-details.css']
})
export class MovieDetails implements OnInit {
  private movieService = inject(MovieService);
  @Input() id!: string;

  // Declaramos un Observable que contendrá TODOS los datos que necesitamos
  movieData$: Observable<{ details: Movie; credits: CreditsResponse }>;

  ngOnInit(): void {
    if (this.id) {
      // forkJoin dispara ambas peticiones al mismo tiempo y crea un objeto con los dos resultados
      this.movieData$ = forkJoin({
        details: this.movieService.getMovieById(this.id),
        credits: this.movieService.getMovieCredits(this.id)
      });
    }
  }

  getBackdropUrl(path: string | null | undefined): string {
    return path ? `https://image.tmbd.org/t/p/original${path}` : '';
  }
}
```

Concepto Clave: ¿Notaste que NO usamos `.subscribe()`? Dejaremos que el HTML se suscriba automáticamente. A esto se le llama usar el **Async Pipe**.

2. El HTML y el Async Pipe

Abre movie-details.html. Usaremos el nuevo control flow @if de Angular junto con el pipe | async.

```
<!-- El pipe '| async' se suscribe automáticamente a movieData$ -->
<!-- 'as data' guarda el resultado en una variable local llamada 'data' -->
@if (movieData$ | async; as data) {
  <div class="details-container">

    <!-- Para acceder a los detalles usamos data.details -->
    <h1 class="movie-title">{{ data.details.title }}</h1>

    <!-- Mostrar Géneros -->
    <div class="genres">
      @for (genre of data.details.genres; track genre.id) {
        <span class="genre-tag">{{ genre.name }}</span>
      }
    </div>

    <!-- Mostrar Elenco usando data.credits -->
    <section class="movie-cast">
      <h2>Elenco Principal</h2>
      <div class="cast-track">
        <!-- El pipe slice:0:15 recorta la lista a los primeros 15 actores -->
        @for (actor of data.credits.cast | slice:0:15; track actor.id) {
          <app-cast-card [actor]="actor"></app-cast-card>
        }
      </div>
    </section>

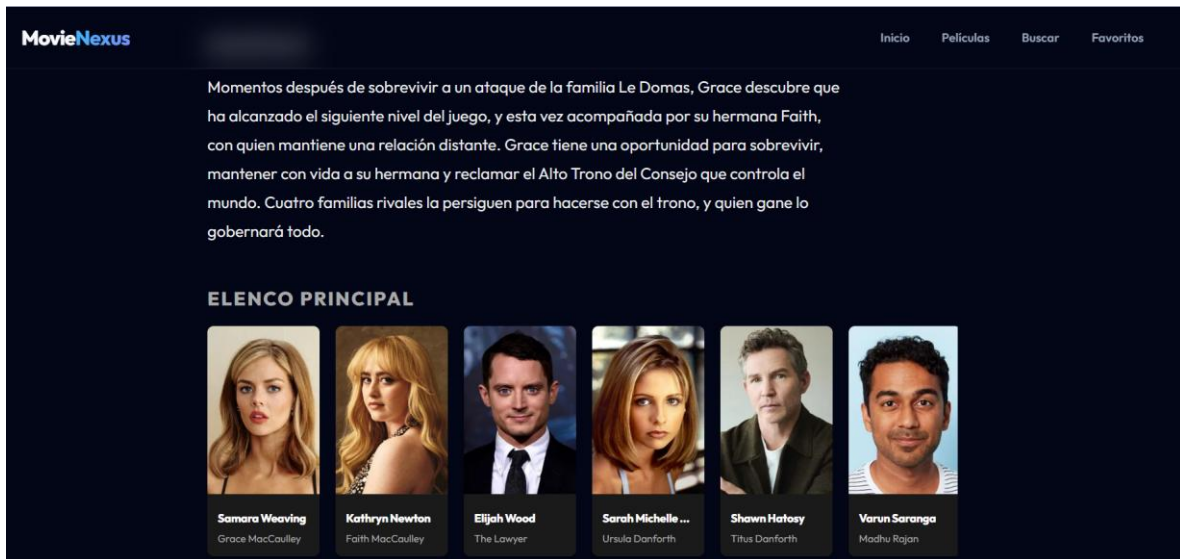
  </div>
} @else {
  <div class="loading-state">Cargando detalles increíbles...</div>
}
```

Prueba de Aprendizaje (Checklist)

1. ¿Para qué sirve el operador forkJoin de RxJS? (Respuesta: Para ejecutar múltiples Observables (peticiones HTTP) en paralelo y esperar a que TODOS terminen para emitir un único resultado combinado. Es ideal para cargar una página que necesita datos de varias APIs).
2. ¿Qué ventaja tiene usar el Async Pipe (| async) en el HTML en lugar de usar .subscribe() en el TypeScript? (Respuesta: El Async Pipe maneja la suscripción automáticamente y, lo más importante, se "desuscribe" (cancela)

automáticamente cuando el componente se destruye, evitando problemas de fugas de memoria).

3. **¿Qué hace el `pipe | slice:0:15` que usamos en el `@for` del `elenco`?** (Respuesta: Toma un array grande y devuelve solo una porción, en este caso desde la posición 0 hasta la 15. Es útil para no saturar la pantalla con 100 actores secundarios).



Control de Versiones

```
git add .
git commit -m "feat: implementar forkJoin, Async Pipe y tarjeta de
actores en detalles de película"
git push origin main
```